# A Software Artifact for Deterministic Conversion of OpenTelemetry Traces into Structured Execution Evidence

Bin Zhang

Independent Researcher, China

## Abstract

This software artifact implements a deterministic transformation from OpenTelemetry trace data into structured execution evidence representations. The repository contains the schema, conversion script, validation path, trace provenance record, baseline comparison, and reproducibility script. The evaluation uses synthetic traces, a public OpenTelemetry trace fixture, and a noisy-trace recovery case to check local conversion and validator behavior. The artifact is designed for reusable inspection of trace-derived execution evidence in scientific software workflows.

## Keywords

OpenTelemetry; tracing; evidence; validation; reproducibility

## Metadata

| Nr. | Code metadata description | Metadata |
|---|---|---|
| C1 | Current code version | v2.1-softwarex-template |
| C2 | Permanent link to code/repository used for this code version | https://github.com/joy7758/agent-evidence/releases/tag/v2.1-softwarex-template |
| C3 | Legal code license | Apache License 2.0 |
| C4 | Code versioning system used | Git |
| C5 | Software code languages, tools, and services used | Python 3, JSON Schema, pytest, OpenTelemetry JSON fixtures |
| C6 | Compilation requirements, operating environments and dependencies | Standard Python environment; install with python3 -m venv .venv and .venv/bin/python -m pip install -e ".[test]" |
| C7 | Developer documentation/manual | README.md, README_SOFTWAREX_SUBMISSION.md, docs/public/EEOAP_v0.1_public_spec.md |
| C8 | Support email for questions | joy7759@gmail.com |

## 1. Motivation and significance

OpenTelemetry traces are widely used to inspect distributed execution, but raw trace JSON is not, by itself, a single validator-readable evidence object. A trace records spans, timing, attributes, and status information. A post-execution review task often needs a different object: one that binds the original trace, selected span identifiers, provenance, evidence references, integrity digests, and a validation result into a compact record that can be checked outside the runtime that produced the trace.

The software described here addresses that gap for a bounded research-software use case. It converts OpenTelemetry Protocol (OTLP) JSON trace data into an Execution Evidence and Operation Accountability Profile (EEOAP) evidence object. In this repository, EEOAP is a versioned, repository-scoped JSON schema and validation target, documented in docs/public/EEOAP_v0.1_public_spec.md and specs/eeoap/v0.1/eeoap.schema.json. This implementation shows how OpenTelemetry trace data can be transformed into an Execution Evidence and Operation Accountability Profile record with preserved identifiers, provenance links, and validator-readable output. The artifact complements OpenTelemetry by targeting validator-readable execution evidence rather than telemetry capture.

The intended users are researchers, tool builders, and developers who already have OpenTelemetry trace exports and want to inspect how a run can be represented as structured execution evidence. The artifact is useful when the question is not only "what spans were observed?" but also "which trace produced this evidence object, what identifiers and digests were preserved, and did the generated object pass the repository validator?"

Related tools such as Jaeger and Zipkin support trace storage and visualization. Logging frameworks record operational events, usually without a single schema-bound evidence object. This artifact complements those tools by producing a reproducible trace-to-evidence mapping for local post-run inspection.

# 2. Software description

## 2.1 Software architecture

The artifact is implemented as a Python research software package with a small conversion and validation surface. The primary entry point for reproducing the paper results is scripts/reproduce_paper.sh:

```
scripts/reproduce_paper.sh
```

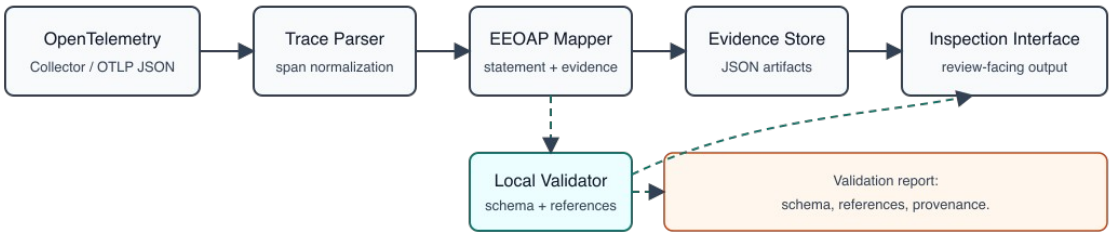


*Figure 1. Software architecture: OpenTelemetry Collector -> Trace Parser -> EEOAP Mapper -> Evidence Store -> Inspection Interface.*

The conversion path has five components:

1. OTLP JSON input fixture: data/otel/raw_demo_trace.json.
2. Trace parser: extracts resource spans, scope spans, trace identifiers, span identifiers, timestamps, attributes, and status fields.
3. EEOAP mapper: writes the trace-derived execution evidence object and adapter record.
4. Evidence output: data/eeoap/real_trace_evidence.json and data/eeoap/real_trace_adapter_record.json.
5. Validation and evaluation: schema checks, baseline comparison, and deterministic-output checks.

The repository also contains a public-facing EEOAP specification, JSON schemas, reproducibility scripts, experiment scripts, and pytest tests. The release archive includes the Python source tree, tests, examples, input fixtures, generated outputs, documentation, and the package manifest.

## 2.2 Software functionalities

The main functions are:

- parse an OpenTelemetry JSON trace fixture;
- preserve trace and span identifiers in a structured output object;
- bind input trace provenance through file references and digests;
- generate an EEOAP evidence object and adapter record;
- validate evidence objects against schema-bound requirements;
- compare raw OTLP retention, generic OTLP JSON export, and EEOAP output;
- reproduce the reported experiments from a clean unpacked archive.

The supported OpenTelemetry surface is intentionally explicit. The current adapter preserves trace ID, span ID, parent span ID, name, kind, timestamps, status code, selected attributes, and instrumentation-scope metadata. Events, links, detailed status semantics, and schema URL metadata are documented as extension points rather than silently treated as fully interpreted fields.

## 2.3 Implementation and interfaces

The conversion interface can be invoked directly:

```
python scripts/convert_otel_trace_to_eeoap.py \
  --input data/otel/raw_demo_trace.json \
  --output data/eeoap/real_trace_evidence.json \
  --adapter-record data/eeoap/real_trace_adapter_record.json
```

The package-level reproduction command runs schema validation, real-trace conversion, baseline comparison, the determinism oracle, the test suite, and table generation:

```
scripts/reproduce_paper.sh
```

The package audit command is:

```
python3 scripts/validate_softwarex_package.py \
  --input release/softwarex_v2_EDITORIAL_LOCKED_FINAL.zip
```

# 3. Illustrative examples

The file examples/io_trace_to_eeoap.json gives a compact input-output example. The input side is an OpenTelemetry JSON trace containing one span chain. The output side is an EEOAP evidence object that records the execution identifier, trace source, span summary, evidence references, status, and schema version.

For the public OpenTelemetry fixture, the reproduction script converts data/otel/raw_demo_trace.json into data/eeoap/real_trace_evidence.json. The generated record exposes the trace identifier, span identifier, input trace digest, output statement digest, and validation status in one place. This lets a developer inspect trace identity, provenance binding, and validator status without manually traversing the raw OTLP JSON.

The evaluation suite contains four executable checks:

| Experiment | Source | Span count | Validator ok | Issue count |
|---|---|---|---|---|
| exp1_synthetic | synthetic fixture | 2 | true | 0 |
| exp2_real_trace | public OpenTelemetry proto example | 1 | true | 0 |
| exp3_noisy_recovery | public example plus trace-format noise | 1 | true | 0 |
| exp4_determinism_oracle | repeated conversion oracle | 1 | true | 0 |

The public OpenTelemetry example is intentionally small and contains a single demonstrated span. It is included as a provenance-tracked real fixture. The synthetic experiment exercises parent-child preservation, and the noisy experiment checks behavior when non-semantic fields are added to the trace input.

# 4. Impact

The artifact supports research and software engineering work that needs reproducible post-execution inspection of trace-derived records. It provides a minimal, executable example of how observability data can be transformed into structured evidence that can be validated, compared, and packaged with its provenance.

The main practical benefit is inspection consistency. Raw trace retention preserves telemetry payloads, but it does not create a single operation-accountability object with policy references, evidence references, provenance links, integrity

digests, and validator-readable output. The adapter adds those surfaces while preserving the original trace as an input artifact.

The baseline comparison in experiments/results/otel_native_vs_eeoap.md distinguishes three representations:

- raw OTLP retention;
- OTLP exported as generic structured JSON;
- EEOAP generation as a validator-readable review artifact.

This comparison clarifies which gains come from retaining telemetry and which gains come from generating a structured evidence object. The artifact can therefore serve as a reusable testbed for future work on trace-derived evidence records, validator design, and reproducible packaging of execution traces.

### 4.1 Evaluation scope

The evaluation focuses on structural correctness and reproducibility within repository-level trace fixtures. The tests are structural rather than performance-oriented. The current package contains one public OTLP JSON example and synthetic/noisy cases for controlled checks. A broader corpus of public multi-span real traces would strengthen future evaluation, but the present artifact already provides a complete executable path from input trace fixture to validated evidence output.

## 5. Conclusions

This SoftwareX artifact provides a reproducible Python implementation for deterministic conversion of OpenTelemetry trace data into structured execution evidence. It includes the conversion code, schemas, examples, real-trace fixture, generated outputs, tests, baseline comparison, architecture figure, and reproduction script. The contribution is a reusable software artifact: it makes the trace-to-evidence mapping inspectable, repeatable, and package-level verifiable.

## Acknowledgements

No specific grant funding was received for this work.

## CRediT author statement

Bin Zhang: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Writing - original draft, Writing - review and editing.

## Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## Data Availability

The software artifact, input trace fixtures, generated EEOAP outputs, evaluation scripts, reproducibility script, and package manifest are available in the public repository release record:

https://github.com/joy7758/agent-evidence/releases/tag/v2.1-softwarex-template

The GitHub release tag identifies the exact package snapshot used for this submission.

## Declaration of Generative AI and AI-assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the author used ChatGPT, Codex, and AI-assisted research tools to support manuscript organization, language refinement, and revision planning. After using these tools, the author reviewed, edited, verified, and adapted the content as needed.

The author takes full responsibility for the content of the submitted manuscript.

## References


1. OpenTelemetry. Trace API specification. https://opentelemetry.io/docs/specs/otel/trace/api/

2. OpenTelemetry. Common specification concepts. https://opentelemetry.io/docs/specs/otel/common/

3. open-telemetry/opentelemetry-proto. examples/trace.json. https://github.com/open-telemetry/opentelemetry-proto/blob/main/examples/trace.json

4. open-telemetry/opentelemetry-proto. opentelemetry/proto/trace/v1/trace.proto. https://github.com/open-telemetry/opentelemetry-proto/blob/main/opentelemetry/proto/trace/v1/trace.proto

5. Jaeger Documentation. https://www.jaegertracing.io/docs/

6. OpenZipkin. https://zipkin.io/

7. Zhang B. Agent Evidence: deterministic OpenTelemetry trace to structured execution evidence conversion. Version v2.1-softwarex-template [software]. GitHub; 2026 Jun 21. https://github.com/joy7758/agent-evidence/releases/tag/v2.1-softwarex-template


## Current executable software version

- Current software version: v2.1-softwarex-template.
- Permanent link to executables of this version: https://github.com/joy7758/agent-evidence/releases/tag/v2.1-softwarex-template.
- Legal software license: Apache License 2.0.
- Computing platform / operating system: macOS/Linux with Python 3.
- Installation requirements and dependencies: pip install -e ".[test]".
- User manual: README_SOFTWAREX_SUBMISSION.md.
- Support email: joy7759@gmail.com.